Linux Sysadmin/Security Basics

By: Sam (elegentair)

Linux Basics Unit

DISCLAIMER: The following 8 slides will be boring, but they provide a needed foundation before we get to the fun parts of Linux, and how computers actually work!

Linux Basics: What is Linux?

Linux is the dominant FOSS (Free and Open Source) OS (Operating System) kernel.

A kernel is the core of an operating system that directly interacts with hardware (think drivers, memory allocation, etc)

The rest of Linux (commands, base applications) is made mostly by the GNU Project (there is some history of how/why it is this way).

Linux is primarily used to power the VAST majority of the internet (servers), all top 500 supercomputers, embedded devices, and is growing in desktop use.

What is Linux (cont.)?

Linux differs significantly from other operating systems (such as Microsoft Windows) in its adoption of UNIX standards and norms, such as:

- Everything is a file
- POSIX Compliance (somewhat)
- Unix philosophy (do one thing and do it well)
- Command line centricity

NOTE: Unix was an operating system invented in 1969 at AT&T Bell Labs. Because they did not have a graphical user interface (they used teletypes that printed on paper, like a command line but on paper), efficient terminal based ways to interact with the system were created.

Basics of the terminal (and POSIX)

The terminal is the center of typical Linux systems. Basic commands are provided by the GNU coreutils project (ls, rm, cd, touch, etc). You can do ANYTHING with a terminal; standard desktop apps use terminal commands in the background to do tasks anyways.

With the terminal you can:

- Edit system config files
- Code
- Manage Files
- Service management
- Any anything else

Basics of the terminal (and POSIX) (cont.)

The way you edit files, issue commands, etc is standard (for the most part) across unix like operating systems (Linux, MacOS, BSD, etc) is mandated by the POSIX standard.

This mandates commands like:

- “rm” command, for removing files and folders
- “touch” command for making new files
- “mkdir” command for making new folders (called directories)
- “ls” for listing the contents of the folder you are in
- “cd” for changing to a new folder (stands for change directory)

It also mandates other factors of the operating system, such as the filesystem:

- Folders are organized like so (if folder 2 is inside folder 1): folder1/folder2
- The root of the filesystem is mounted at /

Disclaimer #2 & #3

#2: This will make sense later, but Linux is not fully POSIX compliant, and POSIX does not mandate everything unique about it. Factors such as much of the file system, the “everything is a file” mentality, etc, is inherited from UNIX (Linux is a UNIX Like OS).

#3: Don’t worry if some of the above language doesn’t make sense to you. It will once we start going over the fun parts!

Everything is a file!

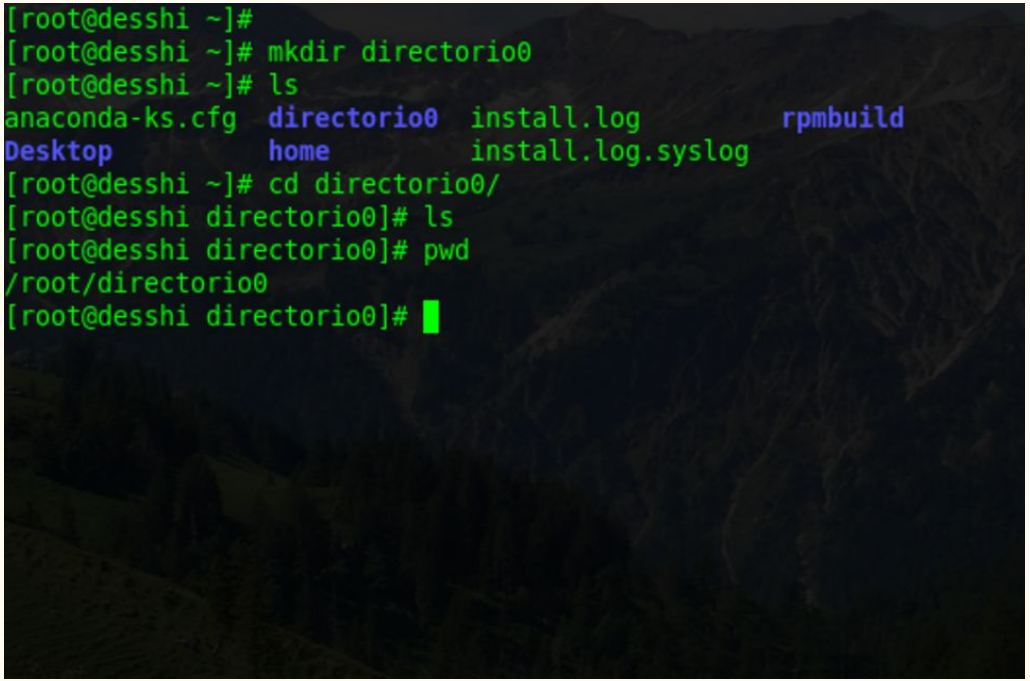
In unix-like operating systems (Linux included), everything is a file! Want to edit settings? Open a configuration file! Want to launch an app? Launch a file!

The following things are represented as files in Linux:

- Applications
- Settings (configuration files)
- System services (depending on your system's init system)
- Drives (hard drives, SSD's, flash drives, floppy driver, etc!)
- And more niche things that we'll go over later

A general overview of the terminal.

This is what a terminal looks like:



NOTE: It does not NECESSARILY look that much like a hacking movie, but this one does with its colors.

The things typed (“ls”, “pwd”, “cd directorio0”) are commands.

Image Attribution: Desshi, CC0, via Wikimedia Commons

A general overview of the terminal (cont.)

Here are some example of terminal commands to get a grasp of what interacting with a terminal looks like:

“pwd”: Prints your current working directory. Remember, directories are folders. So, basically, this command prints the current folder your terminal is in.

“ls”: Lists the contents of your current working directory (the folder you are in)

For example, if I am currently in my user’s (let's call the user “sam”) home directory, my current working directory is “/home/sam”. That is the exact output I get if I type “pwd”.

If I then type “ls”, it will list all the files and folders inside /home/sam.

Finally, if I want to install firefox (a web browser), I can type the following: “sudo apt install firefox”. And it asks for my password, and just does it! No websites, click through installers, or stores required!

Distributions

There are 100s of different Linux distributions (more commonly just called distros).

This is because Linux by itself is not an operating system, it is just a kernel.

Different Linux distributions have different package managers (ways of installing apps), different desktops, init systems, etc.

We will be using the Linux distribution called Debian (and other Linux distributions that were made from debian, also called distributions based on Debian or a Debian derivative).

Debian is not based on any other Linux distribution, and is one of the oldest Linux distros (first released in Sept. 1993). Other distributions that are not based on another are: Slackware (oldest), Gentoo, Void, Arch, openSUSE, Alpine, among other more niche ones.

So what?

All of this is a lot of conceptual information, but as of now it isn’t connected to anything/doesn’t seem to matter.

All of these command line things about Linux make it fun to use, and certainly faster than other computers. I know it looks boring right now, but I PROMISE it isn’t.

Linux Basics Unit done!

Now that you know what makes a Linux system different, lets go over learning commands. We will learn A LOT of them (mostly specific to Debian based distributions).

These will include:

- Basic terminal navigation and files
- Users
- Package Management
- Service Management
- Networks (NEW)
- PAM
- And other relevant Linux-y things throughout this mini course!

P.S: This is where the more fun stuff begins!

File System/Terminal Navigation Unit

File System Basics

Directory: another word for a folder

A linux filesystem has a specific structure. The drive itself has a folder name (called a path or directory) of “/”. From there, there are other directories such as “/etc” or “/bin”. Each user account is stored in the “/home” directory. The main directory for a user “sam” would be “/home/sam/”.

Basic Terminal Navigation/File System Basics

When you are in a terminal, you are always in a certain directory.

By default (when you first open the terminal), it is the home directory of the user you are signed into (/home/{username}).

The directory you are in is called your working directory.

To print your current directory, use the “pwd” command.

“ls” command: Basic Terminal Navigation/File System Basics

To list the contents of your working directory, use the “ls” command, with nothing after it.

For example, if I am in my users home directory, and I want to see the contents of it, I type “ls”.

If you include a directory name after “ls”, it will list the contents of that directory.

For example, if I am in my home directory, and I want to see the contents of the Downloads directory, I will type “ls Downloads”. Pretty simple, right?

“cd” command: Basic Terminal Navigation/File System Basics

To switch into another directory, use the “cd” command.

For example, if I am in my home folder (/home/sam), and I want to go to the “Downloads” folder, I type “cd Downloads”. My new working directory is /home/sam/Downloads.

Now that I am in that Downloads folder, if I want to go back to the folder above it, I can type “cd ..”. Now my working directory is /home/sam again.

“touch” & “mkdir” commands: Basic Terminal Navigation/File System Basics

These two commands can be used to make new files and folders.

“mkdir” stands for make directory. If I am in my home folder and want to make a folder called “projects”, I will type: “mkdir projects”.

If I want to make a directory inside projects called “python”, I can either type from my home folder: “mkdir projects/python” OR cd into projects and type “mkdir python”.

“touch” is used to make new files. For example, if inside my projects/python directory I want to make a text file called “readme.txt”, I will type “touch readme.txt”. If I wanted to make this file from my home folder, I would type “touch projects/python/readme.txt”.

“rm” command: Basic Terminal Navigation/File System Basics

The “rm” command can be used to remove files AND folders.

If I am in my projects/python directory and I want to remove readme.txt, I would type “rm readme.txt” OR (if I wanted to do it from my home directory) I would type “rm projects/python/readme.txt”.

If I am in my projects folder and want to delete the python folder inside of it, I would type “rm -rf python” (or from my home directory: “rm -rf projects/python”). (NOTE: This will nuke everything that was inside that folder. You WILL NOT be able to get it back. There is no trash can in the terminal!

“mv” command: Basic Terminal Navigation/File System Basics

The “mv” command stands for move, and it can be used to move one directory or file into another.

Syntax is: “mv {file or folder to be moved} {directory to move the new file or folder into}”

If I am in my home folder and have a file called “config.conf”, and want to move it into my “.config” folder, I would type “mv config.conf .config/”

“cp” command: Basic Terminal Navigation/File System Basics

The “cp” command stands for copy.

If I have a file called config.conf and I want to copy it into my .config folder, I would type: “cp config.conf .config/”

If I have a folder called Downloads with files in it, and I want to copy it (and call the new copy Downloads_copy), I would type “cp -r Downloads/ Downloads_copy/”.

The “-r” is called a command flag, and stands for recursive. That means it will copy folders and EVERYTHING inside the folders.

Package Management Unit

“apt” and what is a package manager?

We are primarily concerned with debian based Linux distributions. Once upon a time in 1999, the first version of Debian with apt was released. Think of apt as like an app store, but in the command line, and existing long before Apple created the app store (those copy cats!).

With apt, you can update your entire system, install and remove programs (we call them packages, hence the name package manager) and view which ones you have installed.

You do this with the command “apt”.

A quick intermission about “sudo”

Like many commands we run in Linux, “apt” modifies system files (files outside of your users home directory). To get permission for this, we use the “sudo” command BEFORE any command that modifies such files.

For example, if I wanted to create a file called config.txt in my systems “/” directory, I would cd into / and then type “sudo touch config.txt”.

GENERAL RULE: With anything that modifies ANYTHING outside YOUR USER’S home directory, use sudo, or you will get an error.

Sudo runs commands as root without you needing to be logged in as root (which is a security risk)

“apt” command

“apt” downloads packages from repositories. Repositories are online servers hosted by your distribution that contain all compiled packages for your distro.

Configuration for apt, including the sources for these repositories, lives in the “/etc/apt/” directory.

To update your debian system (Ubuntu is a fork of Debian, and Linux Mint is a fork of Ubuntu), type “sudo apt update” to refresh the repositories (essentially check for updates). Then, type “sudo apt upgrade” to actually update new packages.

NOTE: Because you are using sudo, you will be prompted for your password.

“apt” command (cont.)

To install a package in apt (for example, we will install the web browser firefox), type: “sudo apt install firefox”.

To remove firefox (and all of its settings in the form of config files), type “sudo apt remove --purge” (NOTE: if you do not want config files for the package to be removed, omit the purge parameter).

“apt” command (cont.)

To list the applications you have installed, type “apt list --installed”.

To see how many applications you have installed, you can pipe it into another command like so:

“apt list --installed | wc -l”. The “|” operator takes the output of one command and feeds it into another command. The wc command (word count) with the -l flag (line count) counts the number of lines in its input. Because apt outputs a new line for every app it lists, this yields the correct number.

Use the command “sudo apt autoremove” to remove any unneeded dependencies (review what is being removed first, sometimes the system tries to remove critical system components. Remember, YOU are the sysadmin, so YOU are responsible for managing the systems stability.

“apt” command (cont.)

Finally, let’s talk about configuration.

The repositories used are listed in “/etc/apt/sources.list” (or sources.list.d/{something}).

Assure there is a security or updates repository! This is distro specific and we will go over this later!